LynxReaders Platform
Plateforme E-Commerce pour Livres

Votre Nom
Université [Nom de l'Université]

7 mai 2026

Résumé

LynxReaders est une plateforme e-commerce complète qui connecte les lecteurs, les vendeurs et les administrateurs dans un écosystème unifié. Construite avec une architecture microservices moderne, la plateforme offre une navigation fluide des livres, des achats, un suivi de livraison et des recommandations alimentées par l'IA grâce à un système de chatbot intelligent.

Mots-clés : E-Commerce, Microservices, Intelligence Artificielle, Next.js, MongoDB, Architecture Distribuée

Table des matières

1 Introduction **10**
1.1 Contexte du Projet . . . 10
1.2 Objectifs du Projet . . . 10
1.3 Portée du Projet . . . 10
1.4 Structure du Rapport . . . 11
1.5 Méthodologie . . . 11

2 Vue d'ensemble du Projet **12**
2.1 Qu'est-ce que LynxReaders ? . . . 12
2.2 Caractéristiques Principales . . . 12
2.2.1 Système Multi-Rôles . . . 12
2.2.2 Notifications en Temps Réel . . . 12
2.2.3 Chatbot Alimenté par l'IA . . . 12
2.2.4 Traitement de Paiement Sécurisé . . . 12
2.2.5 Intégration E-Delivery . . . 13
2.2.6 Design Réactif . . . 13
2.2.7 Stockage Cloud . . . 13
2.2.8 Notifications Email . . . 13
2.3 Objectifs du Projet . . . 13
2.4 Architecture Générale . . . 13
2.5 Valeur Ajoutée . . . 14
2.5.1 Pour les Clients . . . 14
2.5.2 Pour les Vendeurs . . . 14
2.5.3 Pour les Administrateurs . . . 14

3 Stack Technologique **15**
3.1 Technologies Frontend . . . 15
3.1.1 Next.js 14 . . . 15
3.1.2 React 18 . . . 15
3.1.3 TypeScript . . . 15
3.1.4 TailwindCSS . . . 16
3.2 Technologies Backend . . . 16
3.2.1 Express.js . . . 16
3.2.2 Node.js . . . 16
3.3 Technologies de Base de Données . . . 16
3.3.1 MongoDB . . . 16
3.3.2 Prisma ORM . . . 17

3.3.3 Redis . . . 17
3.4 Intelligence Artificielle et Machine Learning . . . 17
3.4.1 TensorFlow.js . . . 17
3.5 Cloud et Infrastructure . . . 17
3.5.1 Docker . . . 17
3.5.2 Services AWS . . . 18
3.5.3 Firebase . . . 18
3.6 DevOps et CI/CD . . . 18
3.6.1 GitHub Actions . . . 18
3.6.2 Nginx . . . 18
3.7 Architecture Monorepo . . . 19
3.7.1 Turborepo . . . 19
3.8 Technologies Additionnelles . . . 19
3.9 Tableau Récapitulatif . . . 20

4 Architecture Système 21
4.1 Architecture Microservices . . . 21
4.1.1 Composants de l'Architecture . . . 21
4.2 Architecture de Flux de Données . . . 22
4.2.1 Description du Flux . . . 22
4.3 Architecture de Sécurité . . . 22
4.3.1 Authentification . . . 22
4.3.2 Autorisation . . . 22
4.3.3 Chiffrement des Données . . . 22
4.3.4 Sécurité des Mots de Passe . . . 23
4.3.5 Sécurité API . . . 23
4.3.6 Protection des Données Personnelles . . . 23
4.4 Communication Inter-Services . . . 23
4.4.1 Communication Synchrone . . . 23
4.4.2 Communication Asynchrone . . . 23
4.4.3 Gestion des Erreurs . . . 23
4.5 Scalabilité . . . 23
4.5.1 Scalabilité Horizontale . . . 23
4.5.2 Scalabilité Verticale . . . 23
4.5.3 Load Balancing . . . 23
4.6 Disponibilité et Résilience . . . 23
4.6.1 Health Checks . . . 23
4.6.2 Circuit Breaker . . . 24
4.6.3 Retry Logic . . . 24
4.7 Monitoring et Logging . . . 24
4.7.1 Logging Centralisé . . . 24
4.7.2 Métriques . . . 24
4.7.3 Alertes . . . 24
4.8 Diagramme d'Architecture . . . 24
4.9 Avantages de l'Architecture Microservices . . . 24
4.10 Défis et Solutions . . . 24

4.10.1 Défi : Communication Inter-Services . . . 24
4.10.2 Défi : Gestion des Données . . . 25
4.10.3 Défi : Déploiement . . . 25
4.10.4 Défi : Monitoring . . . 25

5 Rôles Utilisateurs et Fonctionnalités 26
5.1 Rôle Utilisateur (Client) . . . 26
5.1.1 Inscription et Authentification Utilisateur . . . 26
5.1.2 Parcours et Recherche de Livres . . . 26
5.1.3 Panier d'Achat et Paiement . . . 27
5.1.4 Gestion des Commandes . . . 27
5.1.5 Avis et Notes . . . 27
5.1.6 Liste de Souhaits et Préférences . . . 28
5.1.7 Interaction avec le Chatbot IA . . . 28
5.2 Rôle Vendeur (Marchand) . . . 28
5.2.1 Inscription et Vérification Vendeur . . . 28
5.2.2 Gestion de l'Inventaire . . . 29
5.2.3 Traitement des Commandes . . . 29
5.2.4 Analyses des Ventes . . . 29
5.2.5 Communication Client . . . 30
5.3 Rôle Administrateur . . . 30
5.3.1 Gestion des Utilisateurs . . . 30
5.3.2 Gestion des Vendeurs . . . 30
5.3.3 Gestion des Livres et Catégories . . . 30
5.3.4 Supervision des Commandes . . . 31
5.3.5 Analyses de la Plateforme . . . 31
5.4 Rôle Centre d'Appel (Intégration Chatwoot) . . . 31
5.4.1 Intégration Chatwoot . . . 31
5.4.2 Fonctions de Support Client . . . 32
5.4.3 Support Multi-Canal . . . 32

6 Intégration E-Delivery 33
6.1 Vue d'ensemble . . . 33
6.2 Architecture d'Intégration . . . 33
6.2.1 Flux d'Intégration . . . 33
6.3 Intégration API E-Delivery . . . 33
6.3.1 Création de Commande . . . 33
6.3.2 Suivi en Temps Réel . . . 34
6.3.3 Intégration Webhook . . . 34
6.4 Fonctionnalités de Gestion de Livraison . . . 34
6.4.1 Pour les Clients . . . 34
6.4.2 Pour les Vendeurs . . . 35
6.4.3 Pour les Administrateurs . . . 35
6.5 Implémentation Technique . . . 35
6.5.1 Endpoints API . . . 35
6.5.2 Schéma de Base de Données . . . 36
6.6 Gestion des Erreurs . . . 37

6.6.1 Erreurs de Création . . . 37
6.6.2 Erreurs de Livraison . . . 37
6.7 Sécurité et Confidentialité . . . 37
6.7.1 Authentification API . . . 37
6.7.2 Protection des Données . . . 37
6.8 Métriques et Analytics . . . 37
6.8.1 Métriques Suivies . . . 37
6.9 Avantages de l'Intégration E-Delivery . . . 38

7 Fonctionnalité Chatbot IA 39
7.1 Vue d'ensemble . . . 39
7.2 Architecture IA . . . 39
7.2.1 Modèle TensorFlow.js . . . 39
7.2.2 Pipeline d'Entraînement du Modèle . . . 40
7.3 Capacités du Chatbot . . . 40
7.3.1 Recommandations de Livres . . . 40
7.3.2 Recherche et Requêtes de Livres . . . 40
7.3.3 Suivi de Commande . . . 40
7.3.4 Requêtes d'Événements et Mises à Jour . . . 40
7.3.5 Notifications en Temps Réel . . . 41
7.4 Implémentation Technique . . . 41
7.4.1 Architecture du Service IA . . . 41
7.4.2 Communication WebSocket . . . 41
7.4.3 Sécurité et Confidentialité . . . 42
7.5 Design de l'Interface Chatbot . . . 42
7.6 Reformation du Modèle . . . 42
7.6.1 Pipeline Automatisé . . . 42
7.6.2 Métriques Suivies . . . 43
7.7 Exemples d'Interactions . . . 43
7.7.1 Exemple 1 : Recommandation Personnalisée . . . 43
7.7.2 Exemple 2 : Recherche de Livres . . . 43
7.8 Performance et Optimisation . . . 44
7.8.1 Optimisations du Modèle . . . 44
7.8.2 Mise à l'Échelle . . . 44

8 Diagrammes UML 45
8.1 Diagramme de Cas d'Utilisation . . . 45
8.1.1 Acteurs . . . 45
8.1.2 Cas d'Utilisation Principaux . . . 45
8.2 Diagramme de Classes . . . 46
8.2.1 Classes Principales . . . 46
8.3 Diagramme de Séquence - Placement de Commande . . . 48
8.3.1 Description du Flux . . . 48
8.4 Diagramme de Séquence - Interaction Chatbot IA . . . 48
8.4.1 Description du Flux . . . 49
8.5 Diagramme d'Activité - Traitement de Commande Vendeur . . . 49
8.5.1 Flux d'Activité . . . 49

8.6 Diagramme de Composants . . . 49
8.6.1 Composants Principaux . . . 50
8.7 Diagramme de Déploiement . . . 50
8.7.1 Infrastructure . . . 50
8.8 Diagramme Entité-Relation (ERD) . . . 50
8.8.1 Entités Principales . . . 50
8.9 Diagramme d'État - Statut de Commande . . . 51
8.9.1 États de Commande . . . 51
8.9.2 Transitions . . . 51
8.10 Diagramme de Collaboration . . . 51
8.10.1 Interactions entre Services . . . 51

9 Outils de Développement 52
9.1 Visual Studio Code . . . 52
9.1.1 IDE Principal . . . 52
9.1.2 Extensions Utilisées . . . 52
9.2 Contrôle de Version . . . 52
9.2.1 Git & GitHub . . . 52
9.2.2 Stratégie de Branches . . . 53
9.2.3 Workflow . . . 53
9.3 Test d'API . . . 53
9.3.1 Outils . . . 53
9.3.2 Endpoints Testés . . . 53
9.4 Gestion de Base de Données . . . 53
9.4.1 MongoDB Compass . . . 53
9.4.2 Prisma Studio . . . 54
9.5 Gestion de Conteneurs . . . 54
9.5.1 Docker Desktop . . . 54
9.5.2 Docker Compose . . . 54
9.6 Outils de Documentation . . . 54
9.6.1 Markdown . . . 54
9.6.2 LaTeX . . . 54
9.7 Outils de Débogage . . . 54
9.7.1 Chrome DevTools . . . 54
9.7.2 Node.js Debugger . . . 55
9.8 Gestion de Projet . . . 55
9.8.1 GitHub Issues . . . 55
9.8.2 GitHub Projects . . . 55
9.9 Outils de Qualité de Code . . . 55
9.9.1 ESLint . . . 55
9.9.2 Prettier . . . 55
9.9.3 Husky . . . 55
9.10 Monitoring et Performance . . . 55
9.10.1 Lighthouse . . . 55
9.10.2 Web Vitals . . . 55
9.11 Outils de Collaboration . . . 55

9.11.1 GitHub . . . 55
9.11.2 Communication . . . 56

10 Déploiement et DevOps **57**
10.1 Infrastructure d'Hébergement . . . 57
10.1.1 Spécifications VPS OVH . . . 57
10.2 Configuration des Domaines . . . 57
10.2.1 Domaines . . . 57
10.2.2 Certificats SSL . . . 57
10.3 Pipeline CI/CD . . . 58
10.3.1 GitHub Actions Workflow . . . 58
10.3.2 Processus Automatisés . . . 58
10.4 Monitoring et Logging . . . 58
10.4.1 Outils . . . 58
10.4.2 Alertes . . . 59
10.5 Stratégie de Sauvegarde . . . 59
10.5.1 Sauvegardes de Base de Données . . . 59
10.5.2 Sauvegardes de Fichiers . . . 59
10.5.3 Sauvegardes de Code . . . 59
10.6 Scalabilité . . . 59
10.6.1 Scalabilité Horizontale . . . 59
10.6.2 Scalabilité Verticale . . . 60
10.6.3 Auto-Scaling . . . 60
10.7 Sécurité . . . 60
10.7.1 Mesures de Sécurité . . . 60
10.8 Configuration Docker . . . 60
10.8.1 Docker Compose . . . 60
10.9 Gestion des Environnements . . . 61
10.9.1 Environnements . . . 61
10.9.2 Variables d'Environnement . . . 61
10.10 Performance et Optimisation . . . 61
10.10.1 Optimisations . . . 61
10.11 Disaster Recovery . . . 61
10.11.1 Plan de Récupération . . . 61
10.12 Métriques et Analytics . . . 62
10.12.1 Métriques Suivies . . . 62

11 Conclusion **63**
11.1 Réalisations du Projet . . . 63
11.2 Points Forts Techniques . . . 63
11.2.1 Excellence Frontend . . . 63
11.2.2 Robustesse Backend . . . 63
11.2.3 Innovation IA . . . 64
11.2.4 Maturité DevOps . . . 64
11.3 Défis Surmontés . . . 64
11.4 Améliorations Futures . . . 64
11.4.1 Fonctionnalités Prévues . . . 64

11.4.2 Améliorations Techniques . . . 65
11.5 Acquis d'Apprentissage . . . 65
11.5.1 Compétences Techniques Acquises . . . 65
11.5.2 Compétences Douces Développées . . . 65
11.6 Impact du Projet . . . 66
11.6.1 Valeur Business . . . 66
11.6.2 Valeur Utilisateur . . . 66
11.6.3 Valeur Technique . . . 66
11.7 Conclusion . . . 66
11.8 Remerciements . . . 67
11.9 Perspectives . . . 67

Bibliographie **68**

A Annexes **71**
A.1 Versions des Technologies . . . 71
A.2 Documentation API . . . 71
A.3 Liens du Dépôt . . . 71
A.4 Membres de l'Équipe . . . 71
A.5 Chronologie du Projet . . . 72
A.6 Structure du Code . . . 72
A.6.1 Structure du Monorepo . . . 72
A.7 Configuration d'Environnement . . . 72
A.7.1 Variables d'Environnement . . . 72
A.8 Endpoints API Principaux . . . 73
A.8.1 Authentification . . . 73
A.8.2 Livres . . . 73
A.8.3 Commandes . . . 74
A.8.4 Paiements . . . 74
A.9 Schémas de Base de Données . . . 74
A.9.1 Schéma User . . . 74
A.9.2 Schéma Book . . . 75
A.10 Commandes Utiles . . . 75
A.10.1 Développement Local . . . 75
A.10.2 Docker . . . 76
A.11 Glossaire . . . 76
A.12 Contact . . . 76

Table des figures

4.1 Architecture Générale de LynxReaders 24

5.1 Page d'inscription utilisateur 26
5.2 Tableau de bord vendeur 28

6.1 Architecture d'intégration E-Delivery 33
6.2 Interface de suivi de livraison 34

7.1 Interface du Chatbot IA 39
7.2 Architecture du Système IA 39
7.3 Exemple de Notification IA 41
7.4 Flux d'Interaction du Chatbot 42

8.1 Diagramme de Cas d'Utilisation - LynxReaders 45
8.2 Diagramme de Classes - Modèle de Données 46
8.3 Diagramme de Séquence - Processus de Commande 48
8.4 Diagramme de Séquence - Interaction Chatbot IA 48
8.5 Diagramme d'Activité - Traitement de Commande par le Vendeur 49
8.6 Diagramme de Composants - Architecture Système 49
8.7 Diagramme de Déploiement - Infrastructure 50
8.8 Diagramme Entité-Relation - Modèle de Données 50
8.9 Diagramme d'État - Cycle de Vie d'une Commande 51
8.10 Diagramme de Collaboration - Communication Inter-Services 51

9.1 Espace de Travail Visual Studio Code 52

10.1 Architecture Serveur 57
10.2 Pipeline CI/CD 58

Liste des tableaux

3.1 Stack Technologique LynxReaders 20

A.1 Versions des Technologies Utilisées 71

Chapitre 1

Introduction

1.1 Contexte du Projet

Le commerce électronique a révolutionné la façon dont les consommateurs achètent des produits, et le secteur du livre n'a pas fait exception. Avec l'augmentation de la demande en ligne pour les livres physiques et numériques, il est devenu essentiel de créer des plateformes qui offrent une expérience utilisateur exceptionnelle tout en répondant aux besoins des différents acteurs de l'écosystème.

LynxReaders a été conçu pour répondre à ce défi en créant une plateforme complète qui connecte les lecteurs, les vendeurs de livres, les administrateurs et les agents de support client dans un environnement unifié et efficace.

1.2 Objectifs du Projet

Les objectifs principaux de ce projet sont :

1. **Créer une plateforme e-commerce scalable** pour la vente de livres avec une architecture moderne et maintenable
2. **Implémenter un système de contrôle d'accès basé sur les rôles** pour différents types d'utilisateurs (clients, vendeurs, administrateurs)
3. **Intégrer des services de livraison tiers** (E-Delivery) pour la gestion des commandes et le suivi en temps réel
4. **Développer un système de recommandation et de support alimenté par l'IA** utilisant TensorFlow.js
5. **Assurer un traitement de paiement sécurisé** et la protection des données
6. **Fournir un suivi de commande en temps réel** et des notifications
7. **Construire des interfaces réactives** pour tous les appareils

1.3 Portée du Projet

Ce projet couvre le développement complet d'une plateforme e-commerce pour livres, incluant :

— Trois applications frontend distinctes (interface utilisateur, tableau de bord vendeur, panneau d'administration)
— Huit microservices backend pour gérer différentes fonctionnalités
— Intégration avec des services tiers (Stripe, E-Delivery, Chatwoot, AWS)
— Système de chatbot IA pour les recommandations et le support
— Infrastructure DevOps avec CI/CD automatisé
— Documentation complète et diagrammes UML

1.4 Structure du Rapport

Ce rapport est organisé en plusieurs chapitres :

— **Chapitre 1** : Introduction et contexte du projet
— **Chapitre 2** : Vue d'ensemble du projet LynxReaders
— **Chapitre 3** : Stack technologique utilisée
— **Chapitre 4** : Architecture système et microservices
— **Chapitre 5** : Rôles utilisateurs et fonctionnalités détaillées
— **Chapitre 6** : Intégration avec E-Delivery
— **Chapitre 7** : Fonctionnalité chatbot IA
— **Chapitre 8** : Diagrammes UML (cas d'utilisation, classes, séquences, etc.)
— **Chapitre 9** : Outils de développement utilisés
— **Chapitre 10** : Déploiement et DevOps
— **Chapitre 11** : Conclusion et perspectives futures

1.5 Méthodologie

Le développement de LynxReaders a suivi une approche agile avec les phases suivantes :

1. **Analyse et Conception** : Définition des exigences, modélisation UML, conception de l'architecture
2. **Développement** : Implémentation itérative des fonctionnalités par microservice
3. **Intégration** : Intégration des services et tests d'intégration
4. **Test** : Tests unitaires, tests d'intégration, tests de performance
5. **Déploiement** : Configuration de l'infrastructure et déploiement en production
6. **Documentation** : Rédaction de la documentation technique et utilisateur

Chapitre 2

Vue d'ensemble du Projet

2.1 Qu'est-ce que LynxReaders ?

LynxReaders est une plateforme e-commerce moderne et complète conçue spécifiquement pour l'industrie du livre. La plateforme sert de marketplace où :

— **Les Lecteurs** peuvent parcourir, rechercher, acheter et suivre les livraisons de livres

— **Les Vendeurs** peuvent lister des livres, gérer l'inventaire et traiter les commandes

— **Les Administrateurs** supervisent les opérations de la plateforme, la gestion des utilisateurs et les analyses

— **Les Agents du Centre d'Appel** fournissent un support client via le système Chatwoot intégré

2.2 Caractéristiques Principales

2.2.1 Système Multi-Rôles

La plateforme offre des interfaces séparées et des fonctionnalités spécifiques pour chaque type d'utilisateur, garantissant une expérience optimale pour chaque rôle.

2.2.2 Notifications en Temps Réel

Système de notifications basé sur WebSocket pour les mises à jour de commandes et les préférences utilisateur.

2.2.3 Chatbot Alimenté par l'IA

Assistant intelligent basé sur TensorFlow.js pour les recommandations personnalisées et le support client.

2.2.4 Traitement de Paiement Sécurisé

Intégration Stripe pour les transactions sécurisées avec support de multiples méthodes de paiement.

2.2.5 Intégration E-Delivery

Intégration avec une entreprise de livraison tierce pour l'exécution des commandes et le suivi en temps réel.

2.2.6 Design Réactif

Approche mobile-first pour toutes les interfaces utilisateur, garantissant une expérience optimale sur tous les appareils.

2.2.7 Stockage Cloud

AWS S3 pour les couvertures de livres et les fichiers multimédias.

2.2.8 Notifications Email

Communication automatisée basée sur SMTP pour les confirmations et mises à jour.

2.3 Objectifs du Projet

1. Créer une plateforme e-commerce scalable pour la vente de livres
2. Implémenter un contrôle d'accès basé sur les rôles pour différents types d'utilisateurs
3. Intégrer des services de livraison tiers (E-Delivery)
4. Développer un système de recommandation et de support alimenté par l'IA
5. Assurer un traitement de paiement sécurisé et la protection des données
6. Fournir un suivi de commande en temps réel et des notifications
7. Construire des interfaces réactives pour tous les appareils

2.4 Architecture Générale

LynxReaders suit une architecture microservices où chaque service est indépendamment déployable et scalable. La plateforme est organisée en :

— **Couche Frontend** : Trois applications Next.js distinctes

— **Couche API Gateway** : Point d'entrée unique pour toutes les requêtes

— **Couche Microservices** : Huit services backend spécialisés

— **Couche Base de Données** : MongoDB pour le stockage principal, Redis pour le cache

— **Services Externes** : Intégrations avec Stripe, E-Delivery, AWS, Chatwoot

2.5 Valeur Ajoutée

2.5.1 Pour les Clients

- Découverte facile de livres avec recherche avancée
- Recommandations personnalisées basées sur l'IA
- Suivi de commande en temps réel
- Support client 24/7 via chatbot et Chatwoot
- Processus de paiement sécurisé

2.5.2 Pour les Vendeurs

- Outils puissants de gestion d'inventaire
- Tableau de bord analytique complet
- Traitement simplifié des commandes
- Intégration automatique avec les services de livraison
- Gestion financière intégrée

2.5.3 Pour les Administrateurs

- Vue d'ensemble complète de la plateforme
- Outils de gestion des utilisateurs et du contenu
- Analytics et rapports détaillés
- Configuration et maintenance simplifiées

Chapitre 3

Stack Technologique

3.1 Technologies Frontend

3.1.1 Next.js 14

Next.js 14 est le framework React utilisé pour construire les interfaces utilisateur.
Caractéristiques utilisées :

- Server-Side Rendering (SSR)
- Static Site Generation (SSG)
- API Routes
- App Router
- Image Optimization

Applications : Trois applications frontend (user-ui, seller-ui, admin-ui)

3.1.2 React 18

React 18 est la bibliothèque de composants UI utilisée.
Caractéristiques utilisées :

- Hooks (useState, useEffect, useContext)
- Context API pour la gestion d'état
- Hooks personnalisés pour la logique réutilisable

3.1.3 TypeScript

TypeScript fournit un JavaScript typé pour une meilleure sécurité de type et une maintenance améliorée.
Avantages :

- Détection d'erreurs à la compilation
- Meilleur support IDE
- Amélioration de la maintenabilité du code

3.1.4 TailwindCSS

TailwindCSS est un framework CSS utilitaire-first pour un développement UI rapide.
Avantages :

- Développement UI rapide
- Système de design cohérent
- Utilitaires de design réactif

3.2 Technologies Backend

3.2.1 Express.js

Express.js est le framework d'application web Node.js utilisé pour construire les microservices.
Services construits :

- API Gateway (Port 9000)
- User Service (Port 3001)
- Seller Service (Port 3002)
- Admin Service (Port 3003)
- Order Service (Port 3004)
- Payment Service (Port 3005)
- Notification Service (Port 3006)
- AI Service (Port 3007)

3.2.2 Node.js

Node.js est l'environnement d'exécution JavaScript utilisé.
Version : 18.x LTS
Avantages :

- I/O non bloquant
- Architecture orientée événements
- Écosystème NPM

3.3 Technologies de Base de Données

3.3.1 MongoDB

MongoDB est la base de données NoSQL principale utilisée.
Hébergement : MongoDB Atlas (Cloud)
Collections principales :

- Users
- Books

— Orders
— Categories
— Reviews
— Preferences
— Conversations

3.3.2 Prisma ORM

Prisma ORM est l'outil de base de données et ORM utilisé.
Caractéristiques utilisées :

— Requêtes de base de données type-safe
— Migrations de schéma
— Seeding de base de données
— Optimisation des requêtes

3.3.3 Redis

Redis est le magasin de données en mémoire utilisé.
Hébergement : Upstash (Cloud)
Cas d'utilisation :

— Gestion de session
— Mise en cache
— Limitation de débit
— Fonctionnalités en temps réel

3.4 Intelligence Artificielle et Machine Learning

3.4.1 TensorFlow.js

TensorFlow.js est la bibliothèque de machine learning pour JavaScript utilisée.
Cas d'utilisation :

— Moteur de recommandation de livres
— Prédiction des préférences utilisateur
— Traitement du langage naturel pour le chatbot
— Analyse de sentiment pour les avis

3.5 Cloud et Infrastructure

3.5.1 Docker

Docker est la plateforme de conteneurisation utilisée.
Avantages :

— Environnements de développement cohérents
— Déploiement facile
— Isolation des services
— Scalabilité

3.5.2 Services AWS

— **S3** : Stockage d'images de couverture de livres et fichiers multimédias
— **CloudFront** : CDN pour les assets statiques
— **EC2** : Option d'hébergement alternative

3.5.3 Firebase

Firebase est utilisé pour les fonctionnalités en temps réel et l'authentification.
Services utilisés :

— Firebase Authentication
— Firebase Cloud Messaging (FCM)
— Firebase Realtime Database (pour le chat)

3.6 DevOps et CI/CD

3.6.1 GitHub Actions

GitHub Actions est utilisé pour le pipeline CI/CD automatisé.
Workflows :

— Tests automatisés sur push
— Construction d'images Docker
— Déploiement vers staging/production
— Renouvellement de certificat SSL

3.6.2 Nginx

Nginx est utilisé comme reverse proxy et load balancer.
Caractéristiques :

— Terminaison SSL/TLS
— Routage de domaine
— Service de fichiers statiques
— Limitation de débit

3.7 Architecture Monorepo

3.7.1 Turborepo

Turborepo est l'outil utilisé pour gérer l'architecture monorepo.
Structure :

Listing 3.1 – Structure du Monorepo

```
LynxReaders/
          apps/
                user-ui/           # Frontend client
                seller-ui/         # Tableau de bord vendeur
                admin-ui/          # Panneau admin
                api-gateway/       # API Gateway
                user-service/      # Gestion utilisateurs
                seller-service/    # Op rations vendeur
                order-service/     # Traitement commandes
                payment-service/   # Gestion paiements
                notification-service/  # Notifications
                ai-service/        # Chatbot IA
          packages/
                shared-types/      # Types TypeScript
                ui-components/     # Composants UI partag s
                utils/             # Fonctions utilitaires
          docker-compose.yml
```

Avantages :

- Partage de code entre services
- Gestion unifiée des dépendances
- Outillage cohérent
- Refactoring plus facile

3.8 Technologies Additionnelles

- **Stripe** : Traitement des paiements
- **Chatwoot** : Plateforme de support client
- **Socket.io** : Communication WebSocket en temps réel
- **JWT** : Tokens d'authentification
- **Bcrypt** : Hachage de mots de passe
- **Nodemailer** : Envoi d'emails
- **Zod** : Validation de schéma

3.9 Tableau Récapitulatif

Table 3.1 – Stack Technologique LynxReaders

Couche	**Technologies**
Frontend	Next.js 14, React 18, TypeScript, TailwindCSS
Backend	Node.js, Express.js, TypeScript
Base de Données	MongoDB, Prisma ORM, Redis
IA/ML	TensorFlow.js
Cloud	AWS S3, Firebase, MongoDB Atlas, Upstash Redis
DevOps	Docker, GitHub Actions, Nginx
Paiements	Stripe
Support	Chatwoot

Chapitre 4

Architecture Système

4.1 Architecture Microservices

LynxReaders suit une architecture microservices où chaque service est indépendamment déployable et scalable.

4.1.1 Composants de l'Architecture

API Gateway (Port 9000)

- Point d'entrée pour toutes les requêtes client
- Routage des requêtes vers les services appropriés
- Limitation de débit et authentification
- Équilibrage de charge

Applications Frontend

- User UI : https://lynxreaders.com
- Seller UI : https://merchant.lynxreaders.com
- Admin UI : https://call-center.lynxreaders.com

Services Backend

- **User Service** : Authentification et profils utilisateur
- **Seller Service** : Gestion des vendeurs et listes de livres
- **Order Service** : Traitement et suivi des commandes
- **Payment Service** : Intégration Stripe
- **Notification Service** : Notifications email et push
- **AI Service** : Chatbot et recommandations

Intégrations Externes

- E-Delivery API : Suivi de livraison
- Stripe API : Traitement des paiements
- Chatwoot : Support client
- AWS S3 : Stockage de fichiers
- MongoDB Atlas : Base de données
- Upstash Redis : Cache

4.2 Architecture de Flux de Données

Le flux de données dans LynxReaders suit le modèle suivant :

Listing 4.1 – Flux de Données

```
Client Request     Nginx     API Gateway     Service Router
   Microservice     Database

                                                External APIs
```

4.2.1 Description du Flux

1. **Requête Client** : L'utilisateur fait une requête depuis l'interface frontend
2. **Nginx** : Reverse proxy qui route la requête vers le bon service
3. **API Gateway** : Point d'entrée centralisé qui authentifie et route la requête
4. **Service Router** : Route la requête vers le microservice approprié
5. **Microservice** : Traite la requête et interagit avec la base de données
6. **Base de Données** : Stocke ou récupère les données nécessaires
7. **APIs Externes** : Si nécessaire, le service communique avec des APIs externes

4.3 Architecture de Sécurité

4.3.1 Authentification

Système basé sur les tokens JWT pour l'authentification sécurisée.

4.3.2 Autorisation

Contrôle d'accès basé sur les rôles (RBAC) pour gérer les permissions.

4.3.3 Chiffrement des Données

SSL/TLS pour les données en transit, chiffrement des données sensibles au repos.

4.3.4 Sécurité des Mots de Passe

Hachage Bcrypt pour le stockage sécurisé des mots de passe.

4.3.5 Sécurité API

Limitation de débit et politiques CORS pour protéger les endpoints.

4.3.6 Protection des Données Personnelles

Masquage et chiffrement des données pour la protection des informations personnelles.

4.4 Communication Inter-Services

4.4.1 Communication Synchrone

Les services communiquent via des appels HTTP REST pour les opérations synchrones.

4.4.2 Communication Asynchrone

WebSockets (Socket.io) pour les communications en temps réel comme les notifications.

4.4.3 Gestion des Erreurs

Système centralisé de gestion des erreurs avec logging et monitoring.

4.5 Scalabilité

4.5.1 Scalabilité Horizontale

Les microservices peuvent être répliqués pour gérer une charge accrue.

4.5.2 Scalabilité Verticale

Possibilité d'augmenter les ressources d'un service spécifique si nécessaire.

4.5.3 Load Balancing

Nginx distribue la charge entre plusieurs instances de services.

4.6 Disponibilité et Résilience

4.6.1 Health Checks

Chaque service expose un endpoint de santé pour le monitoring.

4.6.2 Circuit Breaker

Pattern de circuit breaker pour éviter les cascades d'échecs.

4.6.3 Retry Logic

Logique de retry pour les appels API externes en cas d'échec temporaire.

4.7 Monitoring et Logging

4.7.1 Logging Centralisé

Tous les services envoient leurs logs à un système centralisé.

4.7.2 Métriques

Collecte de métriques de performance pour chaque service.

4.7.3 Alertes

Système d'alertes pour les erreurs critiques et les problèmes de performance.

4.8 Diagramme d'Architecture

[PLACEHOLDER: Ajouter diagramme d'architecture système]

FIGURE 4.1 – Architecture Générale de LynxReaders

4.9 Avantages de l'Architecture Microservices

- **Indépendance** : Chaque service peut être développé et déployé indépendamment
- **Scalabilité** : Services individuels peuvent être mis à l'échelle selon les besoins
- **Technologie** : Possibilité d'utiliser différentes technologies pour différents services
- **Résilience** : Échec d'un service n'affecte pas les autres
- **Maintenance** : Plus facile à maintenir et déboguer

4.10 Défis et Solutions

4.10.1 Défi : Communication Inter-Services

Solution : API Gateway centralisé et protocoles de communication standardisés.

4.10.2 Défi : Gestion des Données

Solution : Base de données partagée avec Prisma ORM pour la cohérence.

4.10.3 Défi : Déploiement

Solution : Docker et Docker Compose pour la conteneurisation et le déploiement simplifié.

4.10.4 Défi : Monitoring

Solution : Logging centralisé et métriques de performance pour chaque service.

Chapitre 5

Rôles Utilisateurs et Fonctionnalités

5.1 Rôle Utilisateur (Client)

Le rôle Utilisateur représente les clients qui parcourent et achètent des livres sur la plateforme.

5.1.1 Inscription et Authentification Utilisateur

Fonctionnalités :

- Inscription par email/mot de passe
- Vérification par email
- Connexion sociale (Google, Facebook)
- Fonctionnalité de réinitialisation de mot de passe
- Gestion de profil

[PLACEHOLDER: Ajouter capture d'écran de l'inscription utilisateur]

FIGURE 5.1 – Page d'inscription utilisateur

5.1.2 Parcours et Recherche de Livres

Fonctionnalités :

- Parcourir les livres par catégorie
- Recherche avancée avec filtres :
  - Genre
  - Auteur
  - Fourchette de prix
  - Date de publication
  - Note
- Page de détails du livre avec :
  - Image de couverture

  - Description
  - Informations sur l'auteur
  - Avis et notes
  - Livres connexes

5.1.3 Panier d'Achat et Paiement

Fonctionnalités :

- Ajouter/retirer des livres du panier
- Mettre à jour les quantités
- Appliquer des codes de réduction
- Voir le résumé de la commande
- Méthodes de paiement multiples :
  - Cartes de crédit/débit (Stripe)
  - Portefeuilles numériques
- Gestion des adresses de livraison

5.1.4 Gestion des Commandes

Fonctionnalités :

- Voir l'historique des commandes
- Suivre les commandes en cours
- Suivi de livraison en temps réel (intégration E-Delivery)
- Notifications de statut de commande :
  - Commande confirmée
  - En traitement
  - Expédiée
  - En livraison
  - Livrée
- Télécharger les factures
- Demander des retours/remboursements

5.1.5 Avis et Notes

Fonctionnalités :

- Noter les livres (1-5 étoiles)
- Écrire des avis détaillés
- Télécharger des images d'avis
- Modifier/supprimer ses propres avis
- Système de vote utile

5.1.6 Liste de Souhaits et Préférences

Fonctionnalités :

- Sauvegarder des livres dans la liste de souhaits
- Définir les préférences de lecture
- Recevoir des recommandations personnalisées
- Recevoir des notifications pour :
  - Baisses de prix
  - Nouvelles sorties dans les genres favoris
  - Articles réapprovisionnés

5.1.7 Interaction avec le Chatbot IA

Fonctionnalités :

- Poser des questions sur les livres
- Obtenir des recommandations personnalisées
- Suivre le statut de la commande via le chat
- Interroger sur les événements et mises à jour
- Compréhension du langage naturel

5.2 Rôle Vendeur (Marchand)

Le rôle Vendeur représente les vendeurs de livres qui listent et vendent des livres sur la plateforme.

5.2.1 Inscription et Vérification Vendeur

Fonctionnalités :

- Soumission d'informations commerciales
- Vérification de documents
- Configuration du compte bancaire pour les paiements
- Création de profil de magasin
- Informations fiscales

[PLACEHOLDER: Ajouter capture d'écran du tableau de bord vendeur]

FIGURE 5.2 – Tableau de bord vendeur

5.2.2 Gestion de l'Inventaire

Fonctionnalités :

- Ajouter de nouveaux livres avec :
  - Titre, auteur, ISBN
  - Description
  - Catégorie/genre
  - Prix et réduction
  - Quantité en stock
  - Téléchargement d'image de couverture (AWS S3)
- Téléchargement en masse via CSV
- Modifier les listes existantes
- Gérer les niveaux de stock
- Définir des alertes de stock faible
- Archiver/supprimer les listes

5.2.3 Traitement des Commandes

Fonctionnalités :

- Voir les commandes entrantes
- Gestion du statut des commandes :
  - Accepter/rejeter les commandes
  - Marquer comme emballé
  - Générer des étiquettes d'expédition
  - Confirmer la remise à E-Delivery
- Filtrage et recherche de commandes
- Impression de bordereaux d'expédition
- Traitement en masse des commandes

5.2.4 Analyses des Ventes

Fonctionnalités :

- Tableau de bord des revenus
- Tendances et graphiques des ventes
- Livres les plus vendus
- Démographie des clients
- Métriques de performance :
  - Taux d'exécution des commandes
  - Temps de livraison moyen
  - Score de satisfaction client
- Exporter des rapports (PDF, Excel)

5.2.5 Communication Client

Fonctionnalités :

- Répondre aux questions des clients
- Gérer les demandes de retour
- Envoyer des mises à jour de commande
- Messagerie promotionnelle
- Intégration avec Chatwoot

5.3 Rôle Administrateur

Le rôle Administrateur a le contrôle total sur la plateforme et gère toutes les opérations.

5.3.1 Gestion des Utilisateurs

Fonctionnalités :

- Voir tous les utilisateurs (clients, vendeurs)
- Vérification et approbation des utilisateurs
- Bannir/suspendre les utilisateurs
- Réinitialiser les mots de passe
- Voir les journaux d'activité des utilisateurs
- Gérer les rôles et permissions des utilisateurs

5.3.2 Gestion des Vendeurs

Fonctionnalités :

- Approuver/rejeter les candidatures de vendeurs
- Vérifier les documents des vendeurs
- Surveiller la performance des vendeurs
- Définir les taux de commission
- Gérer les litiges avec les vendeurs
- Suspendre/résilier les comptes de vendeurs

5.3.3 Gestion des Livres et Catégories

Fonctionnalités :

- Créer/modifier/supprimer des catégories
- Approuver les listes de livres
- Supprimer le contenu inapproprié
- Mettre en vedette des livres sur la page d'accueil
- Gérer les métadonnées des livres
- Opérations en masse

5.3.4 Supervision des Commandes

Fonctionnalités :

— Voir toutes les commandes de la plateforme
— Surveiller le statut des commandes
— Gérer les litiges et remboursements
— Suivre la performance de livraison
— Générer des rapports de commande
— Intervenir dans les commandes problématiques

5.3.5 Analyses de la Plateforme

Fonctionnalités :

— Revenus totaux et croissance
— Métriques d'acquisition d'utilisateurs
— Aperçu de la performance des vendeurs
— Catégories et livres populaires
— Distribution géographique
— Analytics de trafic
— Taux de conversion
— Tableau de bord en temps réel

5.4 Rôle Centre d'Appel (Intégration Chatwoot)

Le rôle Centre d'Appel fournit un support client via la plateforme Chatwoot intégrée.

5.4.1 Intégration Chatwoot

Qu'est-ce que Chatwoot ? Chatwoot est une plateforme de support client open-source qui fournit le chat en direct, le support email et l'intégration des réseaux sociaux.

Fonctionnalités d'intégration :

— Widget de chat intégré sur toutes les pages utilisateur
— Boîte de réception unifiée pour toutes les questions clients
— Chat en temps réel avec les clients
— Accès à l'historique des commandes clients
— Intégration avec la base de données LynxReaders

5.4.2 Fonctions de Support Client

Fonctionnalités :

- Chat en direct avec les clients
- Voir le profil et l'historique des commandes du client
- Aide avec :
  - Suivi de commande
  - Problèmes de paiement
  - Problèmes de compte
  - Recommandations de livres
  - Demandes de retour/remboursement
- Escalader les problèmes à l'admin
- Créer des tickets de support
- Réponses prédéfinies pour les questions courantes

5.4.3 Support Multi-Canal

Canaux :

- Chat en direct sur le site web
- Support email
- Réseaux sociaux (Facebook, Twitter)
- Intégration WhatsApp

Chapitre 6

Intégration E-Delivery

6.1 Vue d'ensemble

LynxReaders s'intègre avec E-Delivery, une entreprise de logistique tierce externe, pour gérer toute l'exécution des commandes et le suivi de livraison. E-Delivery ne fait pas partie de la plateforme LynxReaders mais fonctionne comme un fournisseur de services de livraison indépendant. Cette intégration assure une expédition fiable et un suivi en temps réel pour les clients sans avoir besoin de construire un système de livraison interne.

Note importante : LynxReaders n'a PAS son propre service de livraison ou interface de livraison. Toutes les opérations de livraison sont gérées exclusivement par E-Delivery via l'intégration API.

6.2 Architecture d'Intégration

[PLACEHOLDER: Ajouter diagramme d'intégration E-Delivery]

FIGURE 6.1 – Architecture d'intégration E-Delivery

6.2.1 Flux d'Intégration

Le flux d'intégration suit cette séquence :

Listing 6.1 – Flux d'intégration E-Delivery

```
Order Placed     LynxReaders Order Service     E-Delivery API
   Delivery Assignment

Customer Tracking     LynxReaders Notification     Webhook Updates
```

6.3 Intégration API E-Delivery

6.3.1 Création de Commande

Lorsqu'un client termine le paiement :

1. Les détails de la commande sont envoyés à l'API E-Delivery
2. E-Delivery assigne un agent de livraison
3. Un numéro de suivi est généré
4. Le client reçoit un lien de suivi

6.3.2 Suivi en Temps Réel

Fonctionnalités de suivi :

- Mises à jour de localisation basées sur GPS
- Temps de livraison estimé
- Informations de contact de l'agent de livraison
- Mises à jour de statut :
  - Commande récupérée
  - En transit
  - En livraison
  - Livrée

[PLACEHOLDER: Ajouter capture d'écran du suivi de livraison]

FIGURE 6.2 – Interface de suivi de livraison

6.3.3 Intégration Webhook

E-Delivery envoie des webhooks à LynxReaders pour :

- Changements de statut
- Confirmation de livraison
- Tentatives de livraison échouées
- Retour à l'expéditeur

6.4 Fonctionnalités de Gestion de Livraison

6.4.1 Pour les Clients

- Carte de suivi en temps réel
- Notifications SMS/email
- Sélection de créneau de livraison
- Contacter l'agent de livraison
- Preuve de livraison (photo/signature)

6.4.2 Pour les Vendeurs

- Création d'expédition en masse
- Impression d'étiquettes d'expédition
- Suivre plusieurs commandes
- Métriques de performance de livraison

6.4.3 Pour les Administrateurs

- Surveiller toutes les livraisons
- Gérer les litiges de livraison
- Voir les analytics E-Delivery
- Gérer les zones et tarifs de livraison

6.5 Implémentation Technique

6.5.1 Endpoints API

Les endpoints API utilisés pour l'intégration E-Delivery :

Listing 6.2 – Endpoints API E-Delivery

```
// Cr er une commande de livraison
POST /api/delivery/create
Body: {
  orderId: string,
  pickupAddress: Address,
  deliveryAddress: Address,
  items: Item[],
  weight: number,
  dimensions: Dimensions
}

// Suivre une livraison
GET /api/delivery/track/:id
Response: {
  trackingId: string,
  status: DeliveryStatus,
  currentLocation: Location,
  estimatedDelivery: Date,
  deliveryAgent: Agent
}

// Recevoir les mises    jour de statut
POST /api/delivery/webhook
Body: {
  trackingId: string,
  status: DeliveryStatus,
```

```
  timestamp: Date,
  location: Location
}

// Calculer les tarifs de livraison
GET /api/delivery/rates
Query: {
  from: Address,
  to: Address,
  weight: number
}
```

6.5.2 Schéma de Base de Données

Listing 6.3 – Schéma DeliveryOrder

```
interface DeliveryOrder {
  id: string;
  orderId: string;
  eDeliveryTrackingId: string;
  status: DeliveryStatus;
  pickupAddress: Address;
  deliveryAddress: Address;
  estimatedDelivery: Date;
  actualDelivery?: Date;
  deliveryAgent?: Agent;
  trackingHistory: TrackingEvent[];
}

enum DeliveryStatus {
  PENDING = 'PENDING',
  PICKED_UP = 'PICKED_UP',
  IN_TRANSIT = 'IN_TRANSIT',
  OUT_FOR_DELIVERY = 'OUT_FOR_DELIVERY',
  DELIVERED = 'DELIVERED',
  FAILED = 'FAILED',
  RETURNED = 'RETURNED'
}

interface TrackingEvent {
  status: DeliveryStatus;
  timestamp: Date;
  location?: Location;
  description: string;
}
```

6.6 Gestion des Erreurs

6.6.1 Erreurs de Création

En cas d'échec de création de commande E-Delivery :

- Log de l'erreur
- Notification à l'administrateur
- Notification au client avec statut alternatif
- Retry automatique avec backoff exponentiel

6.6.2 Erreurs de Livraison

Gestion des livraisons échouées :

- Notification automatique au client
- Option de reprogrammation
- Escalade au support client si nécessaire
- Suivi des tentatives multiples

6.7 Sécurité et Confidentialité

6.7.1 Authentification API

- Clés API sécurisées stockées dans les variables d'environnement
- Authentification par token pour les webhooks
- Validation de signature pour les webhooks entrants

6.7.2 Protection des Données

- Chiffrement des adresses sensibles
- Masquage des informations personnelles dans les logs
- Conformité RGPD pour les données de livraison

6.8 Métriques et Analytics

6.8.1 Métriques Suivies

- Temps de livraison moyen
- Taux de réussite de livraison
- Satisfaction client (notes de livraison)
- Zones de livraison les plus fréquentes
- Temps de réponse de l'API E-Delivery

6.9 Avantages de l'Intégration E-Delivery

— **Fiabilité** : Service de livraison professionnel et éprouvé
— **Couverture** : Réseau de livraison étendu
— **Suivi** : Système de suivi GPS en temps réel
— **Coût** : Pas besoin de construire et maintenir une infrastructure de livraison
— **Focus** : LynxReaders peut se concentrer sur son cœur de métier

Chapitre 7

Fonctionnalité Chatbot IA

7.1 Vue d'ensemble

Le chatbot alimenté par l'IA fournit une assistance intelligente aux utilisateurs, offrant des recommandations de livres personnalisées, répondant aux questions et fournissant des notifications en temps réel sur les nouveaux livres correspondant aux préférences de l'utilisateur.

[PLACEHOLDER: Ajouter capture d'écran de l'interface chatbot]

FIGURE 7.1 – Interface du Chatbot IA

7.2 Architecture IA

7.2.1 Modèle TensorFlow.js

Purpose du Modèle :

- Moteur de recommandation de livres
- Prédiction des préférences utilisateur
- Compréhension du langage naturel
- Analyse de sentiment

Données d'Entraînement :

- Historique de navigation utilisateur
- Modèles d'achat
- Notes et avis sur les livres
- Requêtes de recherche
- Préférences de catégorie
- Démographie utilisateur

[PLACEHOLDER: Ajouter diagramme d'architecture IA]

FIGURE 7.2 – Architecture du Système IA

7.2.2 Pipeline d'Entraînement du Modèle

Le pipeline d'entraînement suit cette séquence :

Listing 7.1 – Pipeline d'entraînement

```
Data Collection     Data Preprocessing     Feature Engineering
Model Training     Evaluation     Deployment
```

Caractéristiques Utilisées :

- Âge, localisation, historique de lecture de l'utilisateur
- Genre, auteur, prix, notes du livre
- Modèles temporels (tendances saisonnières)
- Données de filtrage collaboratif
- Caractéristiques basées sur le contenu

7.3 Capacités du Chatbot

7.3.1 Recommandations de Livres

Le chatbot peut répondre à des requêtes telles que :

- "Recommandez des livres similaires à [nom du livre]"
- "Que devrais-je lire ensuite ?"
- "Montrez-moi les bestsellers en [genre]"
- Suggestions personnalisées basées sur l'historique

7.3.2 Recherche et Requêtes de Livres

- "Trouvez des livres de [nom d'auteur]"
- "Livres sur [sujet]"
- "Livres sous 20$"
- Recherche en langage naturel

7.3.3 Suivi de Commande

- "Où est ma commande ?"
- "Suivez la commande #12345"
- "Quand mon livre arrivera-t-il ?"

7.3.4 Requêtes d'Événements et Mises à Jour

- "Nouvelles sorties cette semaine ?"
- "Événements d'auteurs à venir"
- "Promotions actuelles"

7.3.5 Notifications en Temps Réel

Le chatbot peut envoyer des notifications pour :

- Nouveaux livres correspondant aux préférences
- Baisses de prix sur les articles de la liste de souhaits
- Livres réapprovisionnés
- Offres personnalisées

[PLACEHOLDER: Ajouter exemple de notification]

FIGURE 7.3 – Exemple de Notification IA

7.4 Implémentation Technique

7.4.1 Architecture du Service IA

Composants :

- Serveur de modèle TensorFlow.js
- Processeur de langage naturel
- Moteur de recommandation
- Gestionnaire de contexte
- Générateur de réponses

7.4.2 Communication WebSocket

Communication bidirectionnelle en temps réel :

Listing 7.2 – Communication WebSocket

```
// Client se connecte au service IA
socket.connect('wss://lynxreaders.com/ai')

// Envoyer un message
socket.emit('chat:message', {
  text: 'Recommandez des livres de science-fiction'
})

// Recevoir une r ponse
socket.on('chat:response', (response) => {
  // Afficher la r ponse IA
})

// Recevoir des notifications
socket.on('notification:new-match', (book) => {
  // Afficher la notification
})
```

7.4.3 Sécurité et Confidentialité

Protection des Données :

- Pas d'exposition de PII dans les réponses
- Communication chiffrée (WSS)
- Anonymisation des données utilisateur
- Conformité RGPD
- Chiffrement de l'historique des conversations

Contrôle d'Accès :

- Authentification utilisateur requise
- Limitation de débit par utilisateur
- Sanitisation des requêtes
- Prévention des injections SQL

7.5 Design de l'Interface Chatbot

Fonctionnalités :

- Bouton de chat flottant (en bas à droite)
- Fenêtre de chat minimisable
- Historique des messages
- Indicateurs de frappe
- Boutons de réponse rapide
- Support de médias riches (cartes de livres, images)
- Option de saisie vocale
- Support multilingue

[PLACEHOLDER: Ajouter capture d'écran du flux d'interaction chatbot]

FIGURE 7.4 – Flux d'Interaction du Chatbot

7.6 Reformation du Modèle

7.6.1 Pipeline Automatisé

- Collecte de données quotidienne
- Reformation hebdomadaire du modèle
- Tests A/B de nouveaux modèles
- Monitoring de performance
- Rollback automatique en cas de dégradation

7.6.2 Métriques Suivies

- Précision des recommandations
- Taux de clics
- Taux de conversion
- Satisfaction utilisateur
- Temps de réponse

7.7 Exemples d'Interactions

7.7.1 Exemple 1 : Recommandation Personnalisée

Listing 7.3 – Exemple d'interaction

```
Utilisateur: "Je viens de finir '1984' de George Orwell,
             que devrais-je lire ensuite ?"

Chatbot: "Bas  sur votre lecture de '1984', je recommande :
         1. 'Le Meilleur des Mondes' - Aldous Huxley
         2. 'Fahrenheit 451' - Ray Bradbury
         3. 'La Ferme des Animaux' - George Orwell

         Ces livres partagent des th mes similaires de dystopie
         et de critique sociale."
```

7.7.2 Exemple 2 : Recherche de Livres

Listing 7.4 – Exemple de recherche

```
Utilisateur: "Trouvez-moi des livres de science-fiction
             sous 15 euros"

Chatbot: "J'ai trouv  12 livres de science-fiction dans
         votre budget :

             'Dune' - Frank Herbert (14.99   )
             'Fondation' - Isaac Asimov (12.50   )
             'Le Guide du Voyageur Galactique' - Douglas Adams
            (13.99   )

         Voulez-vous voir plus de r sultats ?"
```

7.8 Performance et Optimisation

7.8.1 Optimisations du Modèle

- Quantification du modèle pour réduire la taille
- Mise en cache des recommandations fréquentes
- Traitement par lots pour améliorer le débit
- Utilisation de Web Workers pour le traitement hors ligne

7.8.2 Mise à l'Échelle

- Plusieurs instances du service IA
- Load balancing pour distribuer la charge
- Mise en cache Redis pour les réponses fréquentes
- Base de données optimisée pour les requêtes de recommandation

Chapitre 8

Diagrammes UML

8.1 Diagramme de Cas d'Utilisation

[PLACEHOLDER: Ajouter diagramme de cas d'utilisation]

FIGURE 8.1 – Diagramme de Cas d'Utilisation - LynxReaders

8.1.1 Acteurs

Les acteurs principaux du système sont :

- **Client (Utilisateur)** : Personne qui achète des livres
- **Vendeur (Marchand)** : Personne qui vend des livres
- **Administrateur** : Gère la plateforme
- **Agent du Centre d'Appel** : Fournit le support client
- **Système E-Delivery** : Service externe de livraison
- **Chatbot IA** : Assistant intelligent

8.1.2 Cas d'Utilisation Principaux

- Parcourir les livres
- Acheter des livres
- Suivre les commandes
- Gérer l'inventaire
- Traiter les commandes
- Gérer les tickets de support
- Générer des rapports
- Gérer les utilisateurs
- Entraîner le modèle IA

8.2 Diagramme de Classes

[PLACEHOLDER: Ajouter diagramme de classes]

FIGURE 8.2 – Diagramme de Classes - Modèle de Données

8.2.1 Classes Principales

Classe User

Listing 8.1 – Classe User

```
class User {
  - id: string
  - email: string
  - password: string
  - role: UserRole
  - profile: Profile
  + register()
  + login()
  + updateProfile()
  + resetPassword()
}
```

Classe Book

Listing 8.2 – Classe Book

```
class Book {
  - id: string
  - title: string
  - author: string
  - isbn: string
  - price: number
  - stock: number
  - sellerId: string
  - categoryId: string
  + create()
  + update()
  + delete()
  + updateStock()
}
```

Classe Order

Listing 8.3 – Classe Order

```
class Order {
  - id: string
  - userId: string
  - items: OrderItem[]
  - total: number
  - status: OrderStatus
  - deliveryId: string
  - paymentId: string
  + create()
  + updateStatus()
  + cancel()
  + track()
}
```

Classe Seller

Listing 8.4 – Classe Seller

```
class Seller {
  - id: string
  - businessName: string
  - verified: boolean
  - rating: number
  - userId: string
  + addBook()
  + processOrder()
  + viewAnalytics()
  + updateInventory()
}
```

Classe Admin

Listing 8.5 – Classe Admin

```
class Admin {
  - id: string
  - permissions: Permission[]
  - userId: string
  + manageUsers()
  + approveBooks()
  + viewReports()
  + configurePlatform()
}
```

Classe ChatMessage

Listing 8.6 – Classe ChatMessage

```
class ChatMessage {
  - id: string
  - userId: string
  - message: string
  - response: string
  - timestamp: Date
  - type: MessageType
  + send()
  + getHistory()
  + generateResponse()
}
```

8.3 Diagramme de Séquence - Placement de Commande

[PLACEHOLDER: Ajouter diagramme de séquence pour placement de commande]

FIGURE 8.3 – Diagramme de Séquence - Processus de Commande

8.3.1 Description du Flux

1. Le client sélectionne des livres et les ajoute au panier
2. Le client procède au paiement
3. L'interface utilisateur envoie la requête à l'API Gateway
4. L'API Gateway route vers le Order Service
5. Le Order Service crée la commande
6. Le Order Service communique avec le Payment Service
7. Le Payment Service traite le paiement via Stripe
8. Le Payment Service confirme le paiement
9. Le Order Service met à jour le statut de la commande
10. Le Notification Service envoie un email de confirmation
11. Le Order Service crée une commande E-Delivery
12. E-Delivery génère un numéro de suivi
13. Le client reçoit la confirmation avec le numéro de suivi

8.4 Diagramme de Séquence - Interaction Chatbot IA

[PLACEHOLDER: Ajouter diagramme de séquence pour chatbot]

FIGURE 8.4 – Diagramme de Séquence - Interaction Chatbot IA

8.4.1 Description du Flux

1. L'utilisateur ouvre l'interface du chatbot
2. L'utilisateur envoie un message via WebSocket
3. Le Chatbot UI envoie le message au AI Service
4. Le AI Service traite le message avec le modèle TensorFlow.js
5. Le AI Service interroge MongoDB pour l'historique utilisateur
6. Le moteur de recommandation génère des suggestions
7. Le AI Service génère une réponse
8. La réponse est envoyée via WebSocket au Chatbot UI
9. L'utilisateur reçoit la réponse dans l'interface

8.5 Diagramme d'Activité - Traitement de Commande Vendeur

[PLACEHOLDER: Ajouter diagramme d'activité]

FIGURE 8.5 – Diagramme d'Activité - Traitement de Commande par le Vendeur

8.5.1 Flux d'Activité

1. **Début** : Nouvelle commande reçue
2. Le vendeur examine la commande
3. **Décision** : Accepter ou Rejeter ?
   - Si Rejeter : Notifier le client, Fin
   - Si Accepter : Continuer
4. Emballer les articles
5. Générer l'étiquette d'expédition
6. Créer la commande E-Delivery
7. Marquer comme expédié
8. Mettre à jour le statut de la commande
9. Notifier le client
10. **Fin**

8.6 Diagramme de Composants

[PLACEHOLDER: Ajouter diagramme de composants]

FIGURE 8.6 – Diagramme de Composants - Architecture Système

8.6.1 Composants Principaux

- **Couche Frontend** : Applications Next.js
- **API Gateway** : Point d'entrée centralisé
- **Couche Microservices** : Services backend spécialisés
- **Couche Base de Données** : MongoDB, Redis
- **Services Externes** : Stripe, E-Delivery, AWS
- **Couche IA/ML** : TensorFlow.js

8.7 Diagramme de Déploiement

[PLACEHOLDER: Ajouter diagramme de déploiement]

FIGURE 8.7 – Diagramme de Déploiement - Infrastructure

8.7.1 Infrastructure

- **Serveur VPS OVH** : 51.178.55.5
- **Conteneurs Docker** : Services conteneurisés
- **Reverse Proxy Nginx** : Routage et SSL
- **MongoDB Atlas** : Base de données cloud
- **Redis Upstash** : Cache cloud
- **AWS S3** : Stockage cloud
- **GitHub Actions** : CI/CD

8.8 Diagramme Entité-Relation (ERD)

[PLACEHOLDER: Ajouter diagramme ERD]

FIGURE 8.8 – Diagramme Entité-Relation - Modèle de Données

8.8.1 Entités Principales

- **Users** : (1 :M) avec Orders, Reviews, Conversations
- **Books** : (M :1) avec Sellers, (1 :M) avec Reviews, (M :M) avec Categories
- **Orders** : (M :1) avec Users, (1 :M) avec OrderItems
- **Categories** : (M :M) avec Books
- **Reviews** : (M :1) avec Users, (M :1) avec Books
- **Sellers** : (1 :M) avec Books, (1 :M) avec Orders
- **Conversations** : (M :1) avec Users, (1 :M) avec Messages

Note : Les informations de suivi E-Delivery sont stockées dans la table Orders comme référence externe (eDeliveryTrackingId), pas comme entité séparée.

8.9 Diagramme d'État - Statut de Commande

[PLACEHOLDER: Ajouter diagramme d'état]

FIGURE 8.9 – Diagramme d'État - Cycle de Vie d'une Commande

8.9.1 États de Commande

1. **Pending Payment** : En attente de paiement
2. **Payment Confirmed** : Paiement confirmé
3. **Processing** : En traitement
4. **Packed** : Emballée
5. **Shipped** : Expédiée
6. **Out for Delivery** : En livraison
7. **Delivered** : Livrée
8. **Cancelled** : Annulée
9. **Returned** : Retournée

8.9.2 Transitions

— Pending → Confirmed (succès du paiement)
— Confirmed → Processing (vendeur accepte)
— Processing → Packed (articles emballés)
— Packed → Shipped (remis à E-Delivery)
— Shipped → Out for Delivery (agent de livraison assigné)
— Out for Delivery → Delivered (client reçoit)
— Tout état → Cancelled (avant expédition)
— Delivered → Returned (demande de retour)

8.10 Diagramme de Collaboration

[PLACEHOLDER: Ajouter diagramme de collaboration]

FIGURE 8.10 – Diagramme de Collaboration - Communication Inter-Services

8.10.1 Interactions entre Services

Le diagramme de collaboration montre comment les différents services communiquent entre eux pour accomplir des tâches complexes, comme le traitement d'une commande complète impliquant plusieurs services.

Chapitre 9

Outils de Développement

9.1 Visual Studio Code

9.1.1 IDE Principal

Visual Studio Code (VS Code) est l'environnement de développement intégré principal utilisé pour le développement de LynxReaders.

9.1.2 Extensions Utilisées

- **ESLint** : Linting du code pour maintenir la qualité
- **Prettier** : Formatage automatique du code
- **TypeScript** : Support du langage TypeScript
- **Docker** : Gestion des conteneurs
- **GitLens** : Intégration Git avancée
- **Tailwind CSS IntelliSense** : Autocomplétion pour les utilitaires CSS
- **MongoDB for VS Code** : Gestion de base de données MongoDB
- **Thunder Client** : Test d'API intégré
- **Error Lens** : Affichage inline des erreurs

[PLACEHOLDER: Ajouter capture d'écran de l'espace de travail VS Code]

FIGURE 9.1 – Espace de Travail Visual Studio Code

9.2 Contrôle de Version

9.2.1 Git & GitHub

Dépôt : `github.com/lynxreaders/lynxreaders-app`

9.2.2 Stratégie de Branches

— `master` : Branche de production
— `staging` : Branche de test
— `feature/*` : Branches de fonctionnalités
— `bugfix/*` : Branches de correction de bugs

9.2.3 Workflow

1. Créer une branche de fonctionnalité
2. Développer et committer les changements
3. Pousser vers GitHub
4. Créer une pull request
5. Revue de code
6. Fusionner vers staging
7. Tester sur l'environnement de staging
8. Fusionner vers master
9. Déploiement automatique en production

9.3 Test d'API

9.3.1 Outils

— **Thunder Client** : Extension VS Code pour les tests d'API
— **Postman** : Client API externe
— **cURL** : Commandes en ligne de commande

9.3.2 Endpoints Testés

— Flux d'authentification
— Opérations CRUD
— Traitement des paiements
— Téléchargements de fichiers
— Connexions WebSocket

9.4 Gestion de Base de Données

9.4.1 MongoDB Compass

Fonctionnalités :

— Explorateur de base de données visuel

— Constructeur de requêtes
— Gestion des index
— Monitoring de performance

9.4.2 Prisma Studio

Fonctionnalités :

— Interface graphique de base de données
— Édition de données
— Visualisation des relations

9.5 Gestion de Conteneurs

9.5.1 Docker Desktop

Fonctionnalités :

— Monitoring des conteneurs
— Gestion des images
— Gestion des volumes
— Configuration réseau

9.5.2 Docker Compose

Utilisation :

— Orchestration multi-conteneurs
— Dépendances entre services
— Configuration d'environnement

9.6 Outils de Documentation

9.6.1 Markdown

Utilisé pour la documentation technique et les README.

9.6.2 LaTeX

Utilisé pour la génération de rapports académiques et de documentation formelle.

9.7 Outils de Débogage

9.7.1 Chrome DevTools

Utilisé pour le débogage frontend et l'analyse de performance.

9.7.2 Node.js Debugger

Utilisé pour le débogage backend avec VS Code.

9.8 Gestion de Projet

9.8.1 GitHub Issues

Utilisé pour suivre les bugs, les fonctionnalités et les tâches.

9.8.2 GitHub Projects

Utilisé pour la gestion de projet avec des tableaux Kanban.

9.9 Outils de Qualité de Code

9.9.1 ESLint

Configuration ESLint pour maintenir la cohérence du code et détecter les erreurs.

9.9.2 Prettier

Formatage automatique du code pour maintenir un style cohérent.

9.9.3 Husky

Git hooks pour exécuter des tests et des vérifications avant les commits.

9.10 Monitoring et Performance

9.10.1 Lighthouse

Utilisé pour auditer les performances et l'accessibilité des applications web.

9.10.2 Web Vitals

Métriques de performance web pour mesurer l'expérience utilisateur.

9.11 Outils de Collaboration

9.11.1 GitHub

— Contrôle de version
— Gestion de projet

— Revue de code
— CI/CD

9.11.2 Communication

— Discussions GitHub
— Commentaires sur les pull requests
— Documentation collaborative

Chapitre 10

Déploiement et DevOps

10.1 Infrastructure d'Hébergement

10.1.1 Spécifications VPS OVH

- **Adresse IP** : 51.178.55.5
- **CPU** : 4 cœurs
- **RAM** : 8GB
- **Stockage** : 160GB NVMe SSD
- **OS** : Ubuntu 22.04 LTS
- **Coût** : 11,99€/mois

[PLACEHOLDER: Ajouter diagramme d'architecture serveur]

FIGURE 10.1 – Architecture Serveur

10.2 Configuration des Domaines

10.2.1 Domaines

- `lynxreaders.com` → Frontend Utilisateur
- `merchant.lynxreaders.com` → Tableau de Bord Vendeur
- `call-center.lynxreaders.com` → Panneau Admin
- `backendmaster.lynxreaders.com` → API Gateway (Production)
- `backend-staging.lynxreaders.com` → API Gateway (Staging)

10.2.2 Certificats SSL

- Let's Encrypt (Gratuit)
- Renouvellement automatique via Certbot
- HTTPS forcé sur tous les domaines

10.3 Pipeline CI/CD

10.3.1 GitHub Actions Workflow

Le workflow GitHub Actions suit cette séquence :

Listing 10.1 – Workflow CI/CD

```
Trigger: Push vers master/staging

Ex cuter les Tests

Construire les Images Docker

Pousser vers Docker Hub

SSH vers le Serveur VPS

Tirer les Derni res Images

Ex cuter Docker Compose

Health Check

Notifier l' quipe  (Succ s/ chec )
```

10.3.2 Processus Automatisés

- Linting et formatage du code
- Tests unitaires
- Tests d'intégration
- Construction d'images Docker
- Déploiement de conteneurs
- Migrations de base de données
- Renouvellement de certificat SSL

[PLACEHOLDER: Ajouter diagramme du pipeline CI/CD]

FIGURE 10.2 – Pipeline CI/CD

10.4 Monitoring et Logging

10.4.1 Outils

- **Logs Docker** : Monitoring des conteneurs
- **Logs Nginx** : Logs d'accès/erreur

— **Logs d'Application** : Winston pour le logging
— **Logs de Base de Données** : Logs de requêtes
— **Métriques de Performance** : Monitoring des performances

10.4.2 Alertes

— Indisponibilité du serveur
— Utilisation CPU/mémoire élevée
— Déploiements échoués
— Problèmes de connexion à la base de données
— Erreurs API

10.5 Stratégie de Sauvegarde

10.5.1 Sauvegardes de Base de Données

— Sauvegardes automatiques MongoDB Atlas
— Snapshots quotidiens
— Rétention de 7 jours
— Récupération point-in-time

10.5.2 Sauvegardes de Fichiers

— Versioning AWS S3 activé
— Réplication cross-région
— Politiques de cycle de vie

10.5.3 Sauvegardes de Code

— Dépôt GitHub
— Clones multiples des développeurs
— Releases taguées

10.6 Scalabilité

10.6.1 Scalabilité Horizontale

— Ajouter plus d'instances VPS
— Load balancer (Nginx)
— Sharding de base de données

10.6.2 Scalabilité Verticale

- Mettre à niveau les ressources VPS
- Optimiser les requêtes de base de données
- Implémenter la mise en cache (Redis)

10.6.3 Auto-Scaling

- Docker Swarm ou Kubernetes
- Orchestration de conteneurs
- Allocation dynamique de ressources

10.7 Sécurité

10.7.1 Mesures de Sécurité

- **Firewall** : Configuration UFW sur Ubuntu
- **SSL/TLS** : Certificats pour tous les domaines
- **Authentification** : JWT avec expiration
- **Rate Limiting** : Protection contre les abus
- **CORS** : Configuration stricte des origines
- **Validation** : Validation des entrées utilisateur
- **Chiffrement** : Chiffrement des données sensibles

10.8 Configuration Docker

10.8.1 Docker Compose

Fichier `docker-compose.yml` pour orchestrer tous les services :

Listing 10.2 – Exemple Docker Compose

```
version: '3.8'
services:
  api-gateway:
    image: lynxreaders/api-gateway:latest
    ports:
      - "9000:9000"
    environment:
      - NODE_ENV=production
    depends_on:
      - user-service
      - seller-service

  user-service:
```

```
    image: lynxreaders/user-service:latest
    ports:
      - "3001:3001"
    environment:
      - DATABASE_URL=${DATABASE_URL}
      - REDIS_URL=${REDIS_URL}

  # ... autres services
```

10.9 Gestion des Environnements

10.9.1 Environnements

- **Development** : Environnement local
- **Staging** : Environnement de test
- **Production** : Environnement live

10.9.2 Variables d'Environnement

Gestion sécurisée des variables d'environnement via :

- Fichiers `.env` pour le développement local
- Secrets GitHub pour CI/CD
- Variables d'environnement Docker pour la production

10.10 Performance et Optimisation

10.10.1 Optimisations

- **Caching** : Redis pour la mise en cache
- **CDN** : CloudFront pour les assets statiques
- **Compression** : Gzip pour les réponses HTTP
- **Image Optimization** : Next.js Image Optimization
- **Database Indexing** : Index MongoDB optimisés

10.11 Disaster Recovery

10.11.1 Plan de Récupération

- Sauvegardes automatiques quotidiennes
- Procédures de restauration documentées
- Tests de récupération réguliers
- Monitoring de la santé du système

10.12 Métriques et Analytics

10.12.1 Métriques Suivies

- Temps de réponse API
- Taux d'erreur
- Utilisation des ressources serveur
- Trafic utilisateur
- Taux de conversion
- Performance de la base de données

Chapitre 11

Conclusion

11.1 Réalisations du Projet

LynxReaders livre avec succès une plateforme e-commerce complète pour livres avec les accomplissements suivants :

1. **Système Multi-Rôles** : Interfaces et fonctionnalités distinctes pour utilisateurs, vendeurs, administrateurs et agents de centre d'appel
2. **Architecture Microservices** : Services scalables et maintenables utilisant des technologies modernes
3. **Intégrations Tierces** : Intégration réussie avec E-Delivery, Stripe, Chatwoot et services AWS
4. **Fonctionnalités Alimentées par l'IA** : Chatbot intelligent développé avec TensorFlow.js pour recommandations et support
5. **Capacités en Temps Réel** : Notifications et suivi basés sur WebSocket
6. **Plateforme Sécurisée** : Protection des données, sécurité PII et conformité RGPD
7. **DevOps Automatisé** : Pipeline CI/CD établi avec GitHub Actions
8. **Design Réactif** : Interfaces mobile-first pour tous les types d'utilisateurs

11.2 Points Forts Techniques

11.2.1 Excellence Frontend

— Next.js 14 avec Server-Side Rendering
— TypeScript pour la sécurité de type
— TailwindCSS pour un développement UI rapide
— Design réactif sur tous les appareils

11.2.2 Robustesse Backend

— Microservices Express.js

— MongoDB avec Prisma ORM
— Cache Redis pour la performance
— Authentification JWT
— Limitation de débit et sécurité

11.2.3 Innovation IA

— Moteur de recommandation TensorFlow.js
— Traitement du langage naturel
— Notifications personnalisées en temps réel
— Reformation continue du modèle

11.2.4 Maturité DevOps

— Conteneurisation Docker
— Pipeline CI/CD automatisé
— Infrastructure as Code
— Monitoring et logging

11.3 Défis Surmontés

1. **Communication Microservices** : Implémentation d'API Gateway pour un routage efficace des services
2. **Fonctionnalités Temps Réel** : Utilisation de WebSockets pour des mises à jour instantanées
3. **Entraînement Modèle IA** : Collecte et traitement de données utilisateur pour des recommandations précises
4. **Intégration Tierce** : Intégration réussie de l'API E-Delivery avec gestion des webhooks
5. **Sécurité** : Implémentation de mesures de sécurité complètes pour protéger les données utilisateur
6. **Scalabilité** : Architecture conçue pour gérer une base d'utilisateurs croissante

11.4 Améliorations Futures

11.4.1 Fonctionnalités Prévues

1. Applications mobiles (iOS et Android)
2. Support e-book et audiobook
3. Service d'abonnement pour lecture illimitée
4. Portail auteur pour l'auto-publication

5. Fonctionnalités sociales (clubs de lecture, discussions de livres)
6. Analytics avancés avec machine learning
7. Support multilingue
8. Option de paiement par cryptomonnaie
9. Aperçus de livres en réalité augmentée
10. Recherche de livres par voix

11.4.2 Améliorations Techniques

1. Kubernetes pour l'orchestration de conteneurs
2. API GraphQL aux côtés de REST
3. Cache côté serveur avec CDN
4. Progressive Web App (PWA)
5. Mesh de microservices avec Istio
6. Monitoring avancé avec Prometheus/Grafana
7. Elasticsearch pour une meilleure recherche
8. File d'attente de messages (RabbitMQ/Kafka)

11.5 Acquis d'Apprentissage

11.5.1 Compétences Techniques Acquises

— Conception d'architecture microservices
— Développement full-stack avec frameworks modernes
— Conception et optimisation de base de données
— Intégration IA/ML dans applications web
— Pratiques DevOps et CI/CD
— Services cloud (AWS, MongoDB Atlas)
— Conteneurisation Docker
— Conception et intégration API
— Meilleures pratiques de sécurité
— Protocoles de communication en temps réel

11.5.2 Compétences Douces Développées

— Planification et gestion de projet
— Résolution de problèmes et débogage
— Documentation de code
— Collaboration d'équipe (workflow Git)
— Gestion du temps
— Rédaction technique

11.6 Impact du Projet

11.6.1 Valeur Business

- Connecte efficacement les lecteurs avec les vendeurs
- Réduit les coûts opérationnels grâce à l'automatisation
- Fournit des insights basés sur les données pour les décisions business
- Améliore l'expérience client avec recommandations IA
- Scalable pour accommoder la croissance business

11.6.2 Valeur Utilisateur

- Découverte et achat de livres faciles
- Suivi de commande en temps réel
- Recommandations personnalisées
- Support chatbot IA 24/7
- Traitement de paiement sécurisé
- Livraison rapide via intégration E-Delivery

11.6.3 Valeur Technique

- Architecture microservices réutilisable
- Infrastructure scalable
- Codebase maintenable
- Pipeline de déploiement automatisé
- Documentation complète

11.7 Conclusion

LynxReaders représente une plateforme e-commerce moderne et complète construite avec des technologies de pointe et des meilleures pratiques. Le projet démontre avec succès l'intégration de multiples systèmes complexes incluant l'architecture microservices, les capacités IA/ML, les APIs tierces et les fonctionnalités en temps réel.

La plateforme sert de solution complète pour l'industrie du livre, fournissant de la valeur à tous les acteurs : les clients profitent d'expériences d'achat personnalisées, les vendeurs gagnent des outils puissants pour la gestion business, les administrateurs ont une supervision complète, et les agents de centre d'appel peuvent fournir un support efficace.

Grâce à ce projet, nous avons acquis une expérience inestimable en développement full-stack, infrastructure cloud, intégration IA et pratiques DevOps modernes. L'architecture modulaire assure que la plateforme peut évoluer avec les exigences changeantes et scaler pour répondre à une demande croissante.

LynxReaders n'est pas seulement un projet universitaire mais une plateforme prête pour la production qui peut servir de vrais utilisateurs et entreprises dans l'industrie du livre.

11.8 Remerciements

Nous tenons à remercier tous ceux qui ont contribué à la réalisation de ce projet, notamment :

— Notre encadrant universitaire pour ses conseils et son orientation
— L'équipe de développement pour leur travail acharné
— Les testeurs bêta pour leurs retours précieux
— La communauté open-source pour les outils et bibliothèques utilisés

11.9 Perspectives

L'avenir de LynxReaders est prometteur avec de nombreuses opportunités d'expansion et d'amélioration. La plateforme est conçue pour évoluer et s'adapter aux besoins changeants du marché du livre, tout en maintenant les plus hauts standards de qualité, sécurité et expérience utilisateur.

Bibliographie

Bibliographie

[1] Next.js Documentation. *Official Next.js Documentation*. https://nextjs.org/docs. Consulté en 2024.

[2] Express.js Guide. *Official Express.js Documentation*. https://expressjs.com. Consulté en 2024.

[3] MongoDB Manual. *MongoDB Official Documentation*. https://docs.mongodb.com. Consulté en 2024.

[4] TensorFlow.js. *TensorFlow.js Official Documentation*. https://www.tensorflow.org/js. Consulté en 2024.

[5] Docker Documentation. *Official Docker Documentation*. https://docs.docker.com. Consulté en 2024.

[6] Richardson, Chris. *Microservices Patterns : With Examples in Java*. Manning Publications, 2018.

[7] AWS Best Practices. *AWS Architecture Center*. https://aws.amazon.com/architecture. Consulté en 2024.

[8] Stripe API Reference. *Stripe Official Documentation*. https://stripe.com/docs/api. Consulté en 2024.

[9] Prisma Documentation. *Official Prisma Documentation*. https://www.prisma.io/docs. Consulté en 2024.

[10] Redis Documentation. *Official Redis Documentation*. https://redis.io/docs. Consulté en 2024.

[11] TypeScript Handbook. *Official TypeScript Documentation*. https://www.typescriptlang.org/docs. Consulté en 2024.

[12] React Documentation. *Official React Documentation*. https://react.dev. Consulté en 2024.

[13] Tailwind CSS Documentation. *Official Tailwind CSS Documentation*. https://tailwindcss.com/docs. Consulté en 2024.

[14] GitHub Actions Documentation. *Official GitHub Actions Documentation*. https://docs.github.com/en/actions. Consulté en 2024.

[15] Nginx Documentation. *Official Nginx Documentation*. https://nginx.org/en/docs. Consulté en 2024.

[16] Chatwoot Documentation. *Official Chatwoot Documentation*. https://www.chatwoot.com/docs. Consulté en 2024.

[17] Node.js Documentation. *Official Node.js Documentation*. https://nodejs.org/docs. Consulté en 2024.

[18] JWT.io. *JSON Web Token Introduction*. https://jwt.io/introduction. Consulté en 2024.

[19] WebSocket API. *MDN Web Docs*. https://developer.mozilla.org/en-US/docs/Web/API/WebSocket. Consulté en 2024.

[20] Fielding, Roy T. *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine, 2000.

Annexe A

Annexes

A.1 Versions des Technologies

Table A.1 – Versions des Technologies Utilisées

Technologie	Version
Node.js	18.x LTS
Next.js	14.x
React	18.x
Express.js	4.x
MongoDB	6.x
Prisma	5.x
TensorFlow.js	4.x
Docker	24.x
TypeScript	5.x

A.2 Documentation API

La documentation complète de l'API est disponible à : https://backendmaster.lynxreaders.com/api-docs

A.3 Liens du Dépôt

— **Dépôt Principal** : github.com/lynxreaders/lynxreaders-app
— **Documentation** : github.com/lynxreaders/project-plan/

A.4 Membres de l'Équipe

[À compléter avec les noms et rôles des membres de l'équipe]

A.5 Chronologie du Projet

[À compléter avec la chronologie du projet et les jalons]

A.6 Structure du Code

A.6.1 Structure du Monorepo

Listing A.1 – Structure Complète du Projet

```
LynxReaders/
          apps/
                user-ui/
                      src/
                      public/
                      package.json
                      next.config.js
                seller-ui/
                admin-ui/
                api-gateway/
                user-service/
                seller-service/
                order-service/
                payment-service/
                notification-service/
                ai-service/
          packages/
                shared-types/
                ui-components/
                utils/
          docker-compose.yml
          package.json
          turbo.json
          README.md
```

A.7 Configuration d'Environnement

A.7.1 Variables d'Environnement

Listing A.2 – Exemple de Variables d'Environnement

```
# Database
DATABASE_URL=mongodb+srv://...
REDIS_URL=redis://...

# AWS
```

```
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
AWS_S3_BUCKET=lynxreaders-media

# Stripe
STRIPE_SECRET_KEY=sk_live_...
STRIPE_PUBLISHABLE_KEY=pk_live_...

# E-Delivery
EDELIVERY_API_KEY=...
EDELIVERY_API_URL=https://api.edelivery.com

# JWT
JWT_SECRET=...
JWT_EXPIRES_IN=7d

# Email
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=...
SMTP_PASS=...

# Frontend URLs
NEXT_PUBLIC_API_URL=https://backendmaster.lynxreaders.com
NEXT_PUBLIC_STRIPE_KEY=pk_live_...
```

A.8 Endpoints API Principaux

A.8.1 Authentification

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/logout
- POST /api/auth/refresh
- POST /api/auth/forgot-password
- POST /api/auth/reset-password

A.8.2 Livres

- GET /api/books
- GET /api/books/ :id
- GET /api/books/search
- GET /api/books/category/ :categoryId
- POST /api/books (Vendeur/Admin)

— PUT /api/books/ :id (Vendeur/Admin)
— DELETE /api/books/ :id (Vendeur/Admin)

A.8.3 Commandes

— GET /api/orders
— GET /api/orders/ :id
— POST /api/orders
— PUT /api/orders/ :id/status
— GET /api/orders/ :id/track

A.8.4 Paiements

— POST /api/payments/create-intent
— POST /api/payments/confirm
— GET /api/payments/ :id

A.9 Schémas de Base de Données

A.9.1 Schéma User

Listing A.3 – Schéma User

```
model User {
  id             String   @id @default(auto()) @map("_id")
     @db.ObjectId
  email          String   @unique
  password       String
  role           UserRole
  profile        Profile?
  createdAt      DateTime @default(now())
  updatedAt      DateTime @updatedAt
  orders         Order[]
  reviews        Review[]
  conversations  Conversation[]
}

enum UserRole {
  USER
  SELLER
  ADMIN
  CALL_CENTER
}
```

A.9.2 Schéma Book

Listing A.4 – Schéma Book

```
model Book {
  id          String   @id @default(auto()) @map("_id")
     @db.ObjectId
  title       String
  author      String
  isbn        String   @unique
  price       Float
  stock       Int
  sellerId    String   @db.ObjectId
  categoryId  String   @db.ObjectId
  coverImage  String
  description String
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  seller      Seller   @relation(fields: [sellerId], references:
     [id])
  category    Category @relation(fields: [categoryId], references:
     [id])
  orderItems  OrderItem[]
  reviews     Review[]
}
```

A.10 Commandes Utiles

A.10.1 Développement Local

Listing A.5 – Commandes de Développement

```
# Installer les d pendances
npm install

# D marrer en mode d veloppement
npm run dev

# Construire pour la production
npm run build

# Ex cuter les tests
npm test

# Linting
npm run lint

# Formatage
```

```
npm run format
```

A.10.2 Docker

Listing A.6 – Commandes Docker

```
# Construire les images
docker-compose build

# D marrer les services
docker-compose up

# D marrer en arri re-plan
docker-compose up -d

# Voir les logs
docker-compose logs -f

# Arr ter les services
docker-compose down
```

A.11 Glossaire

API Gateway Point d'entrée centralisé pour toutes les requêtes API

Microservice Service indépendant et déployable qui gère une fonctionnalité spécifique

WebSocket Protocole de communication bidirectionnel en temps réel

JWT JSON Web Token, standard pour l'authentification

ORM Object-Relational Mapping, technique de programmation

SSR Server-Side Rendering, rendu côté serveur

CI/CD Continuous Integration/Continuous Deployment

RBAC Role-Based Access Control, contrôle d'accès basé sur les rôles

PII Personally Identifiable Information, informations personnelles identifiables

CDN Content Delivery Network, réseau de distribution de contenu

A.12 Contact

Pour toute question ou suggestion concernant le projet LynxReaders :

- Email : [À compléter]
- GitHub : github.com/lynxreaders
- Site Web : https://lynxreaders.com